

Resumen Tc II

Pasos de un programa

.C → compilador → Ensamblador → Enlazador → Cargador → Mem

- Compilador:

- gcc

- traduce de un lenguaje a otro, alto nivel a una forma simbólica

- Fronts ends y back ends

↳ ARM, IA-64, Power PC, x86, x86-64

- Ensamblador

Convierte el lenguaje de bajo nivel a archivo objeto con binarios transforma mnemónicos (ADD) a sus binarios.

Siempre se realizan 2 pasadas de código:

1ra: se crean direcciones a las instrucciones, se encuentran símbolos (etiquetas y variables globales) y se crea una tabla con las direcciones y los símbolos

2da: Se genera el código máquina (.o), este contiene el código máquina e info adicional

Partes del .o:

1) Object file header

es un índice, tipo de org, tamaño total, y donde se encuentran las demás secciones

2) Text segment

el código máquina, dentro del mismo están las resultantes de las instrucciones que fueron pasadas a binario

3) Static data segment

Datos inicializados & Datos no inicializados (BSS) (espacio reservado para variables globales que inician en 0)

4) Symbol table

Contiene nombres de func y variables globales que pueden ser utilizados por otros archivos

5) Relocation information

Direcciones sin resolver → Lista de direcciones que dependen de una dirección que el ensamblador no conoce para que sean reemplazadas

6) Debugging information

relación entre el .c y las instrucciones para poder usar el GDB

- Linking / Enlazado

junta los .o en un solo archivo, reubica espacios de memoria para que no haya superposición y usó info de la table de símbolos para hacer este proceso

1) Colocación de secciones y resolución de símbolos

toma todos los .o, los combina, identifica variables, funciones y que etiquetas se refieren a que partes de código

2) Resolución de referencias internas y 3) Relocalización

Rellena los espacios de memoria que no estén resueltos por el compilador

Asigna direcciones de memoria absoluta a cada sección basándose en el hardware

El resultado tmb es un punto objeto pero con todas las referencias resueltas

Cargador

Coloca el ejecutable en la memoria principal.

- Lee el encabezado para determinar el tamaño de datos y código

- Crea un espacio de direccionamiento acorde al programa

- Copia instrucciones y datos en la memoria

- Inicializa registros

- Salta a la rutina de inicio

Mapa de memoria

S&I/O
Stack
Dyn Data
Heap
Global Data
Text
Exceptions

ARM 32 bits

• 32 bits

• 2^{32} bytes (4 GB)

• direcciones múltiplo de 4
o a 0xFFFFFFFFC

• y los 4 segmentos

- Text

Solo R.O, incluye el código máquina

- Global data

Segmento R.W, guarda las variables son ubicadas en la memoria antes que inicie el programa

ARM → R9

- Dynamic data

Posee el stack y el heap, este almacena las variables generadas por malloc, crece de forma inversa al stack.

- Excepciones, OS y I/O

No son parte del programa, la parte más baja es para el vector de excepciones y como se manejan.
La parte más alta para el SO y los I/O

Linker Script

↳ Archivo de Config para nuestro hardware

Contiene: límites físicos, secciones, creación de símbolos (-sdata, -ssstack)

Cortex-M3 → Flash: 0x08000000 (64 KB)
→ RAM : 0x20000000 (20 KB)

MEMORY{

```
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

ARM busca la tabla de vectores al inicio de la Flash. Por eso esa sección debe ser la primera

SECTIONS{

```
.text : {
    KEEP(*(.isr_vector)) Tabla de vectores primero
    *(.text)
} > FLASH
```

...

Relación con crt.s

- Inicializa el hardware del micro
- define etiquetas del Linker
- Inicializa memoria y copia variables preinicializadas usando las etiquetas del Linker

El micro necesita saber donde inicia su pila (Stack), el código es el siguiente (2 pasos)

1) $_esstack = \text{ORIGIN}(\text{RAM}) + \text{LENGTH}(\text{RAM})$ *cálculo del script*

2) A esta posición de arranque se le establece almacenando en la primera dirección de la Flash, el valor de inicio de la pila.

en el crt.s se asigna $_esstack$ en la primera entrada de tablas de vectores

Variables globales $\longrightarrow$ se guardan en ram (para poder cambiar)
 $\longrightarrow$ en Flash (Para no perder el valor inicial)

LMA: Load Memory Address, lugar donde se guarda el dato

VMA: Virtual Memory Address, dirección donde se asignan los punteros

$_sdata = \text{LOADADDR}(\text{.data});$ *Lee el valor de LMA donde se guarda .data*
 $_data: \{$

$_sdata = .;$

$*(.data);$

$_edata = .;$

$\} > \text{RAM AT} > \text{FLASH}$ */asigne espacio en RAM y guarda en FLASH**

Sección .bss

Variables que valen 0 al arrancar, solo se reserva espacio en RAM

$_bss: \{$

$_sbss = .;$

$*(.bss)$

$_ebss = .;$

$\} > \text{RAM}$

usamos $_ALIGN(4);$ porque así el cortex-M3 es más eficiente

Startup (Necesario en contextos

crt.s o startup.s (tmb puede ser .c), se encarga de

1) Establecer el punto de entrada (Reset Vector)

2) Inicializar el stack pointer

3) Inicializar .data

4) Limpiar a BSS

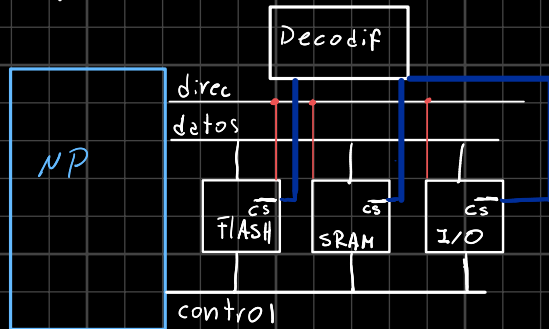
5) configuración del hardware

6) Salto final al main

Periféricos

- Decodificador de direcciones

En todo sistema el bus de direcciones del micro procesador está conectado a todos los dispositivos simultáneamente (SRAM, FLASH, I/O) para evitar que todos respondan al mismo tiempo se implementa un decodificador físico (Red de compuertas lógicas), el mapeo es un proceso de decodificación de hardware, el micro coloca una dirección en el bus y el deco interpreta los bits de mayor peso para activar el chip select correcto.



- Entrada Salida

Una porción de la memoria se dedica a dispositivos entrada salida esto quiere decir que hay ciertas áreas de la memoria (registros) que se le asigna la comunicación directa con el hardware.

Una escritura a una dirección de memoria asignada a un dispositivo indica un envío de datos y una lectura en esa dirección de memoria indica una lectura de datos

```
#define PER_ADDR 0x20FF0000 // write
*(volatile uint32_t *)PER_ADDR = 0xF
```

```
#define PER_ADDR 0x20FF0000 // read
uint32_t valor;
valor = *(volatile uint32_t *)PER_ADDR = 0xF
```

En ARM los periféricos están mapeados en memoria (comparten el mismo espacio en el silicio, no se requieren instrucciones especiales), esto se diseñó así para usar un solo conjunto de instrucciones, flexibilidad de punteros y ortogonalidad)

Registros

Físicamente funcionan como flip flops conectados al chip o a lógica interna

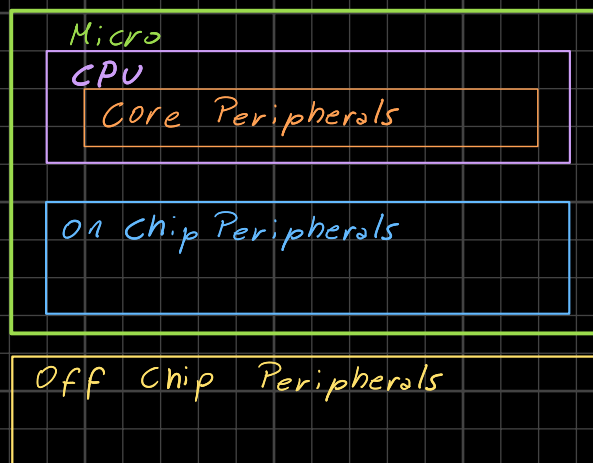
Periféricos

- Externos al núcleo (del fabricante): 0x40000000 a 0x5FFFFFFF
- Se accede mediante APB o AHB
- Pines GPIO, protocolos de comunicación
- Específicos de la blue pill

- Internos (ARM-cortexM3): se comunican mediante el PPB
 - desde 0xE0000000 a 0xE00FFFFF
 - el bus es privado, no compete con el DMA o el USB, el bus posee 32 bits
 - Posee al (SCS, NVIC, SYSTICK, MPU)

Los Buses se separan por 3 razones

- Determinismo y Velocidad
- Seguridad y privilegios
- Portabilidad

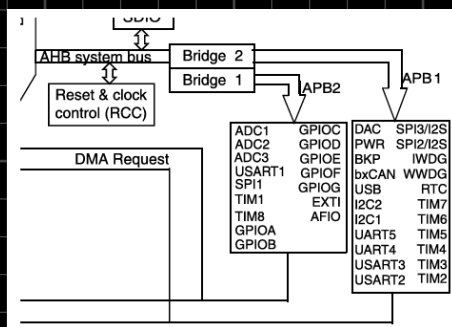


Secuencia de activación

1) Capa de infraestructura

En el ST32, los periféricos se encuentran mapeados en 2 buses

- APB1: 36 MHz
- APB2: 72 MHz



Al comienzo todos los periféricos se encuentran para ahorrar energía, para activarlo se siguen 2 pasos

- 1) Identificar el bus que está vinculado al periférico
- 2) Activar el reloj en el registro correspondiente según el bus

RCC → APB2ENR 1 = (RCC_APB2ENR_USART1EN |
RCC_APB2ENR_ADC1EN);

// Habilita el reloj del puerto ADC1 y USART1

2) Cape de interfaz

Una vez con reloj se asigna un pin físico de I/O
la GPIO → puente físico entre el dispositivo y los pines, AFIO → remapeo de pines
→ *tmb el reloj del GPIO*

Configuramos el modo de uso GPIOX_CRL & GPIOX_CRH, (pines del 0 al 7 y del 8 al 15) y si el periférico necesita pines que no son los convencionales se configura AFIO_MAPR.

ADC → PA1 y PA2 ; UART1_Tx → PA9
UART1_Rx → PA10

// Código en C

RCC → APB2ENR |= RCC_APB2ENR_IOPAEN;

// ADC y UART se encuentran en el puerto A, en el registro
RCC → APB2ENR que contiene todos los puertos

Ahora se configura el modo

- Input floating
- Input pull-up
- Input Pull-down
- analog
- output open-drain
- output push-pull
- Alternate function push-pull
- Alternate function open-drain

Modo de configuración	CNF1	CNF0	MODE1	MODE0	PxODR R.
Salida de Propósitos Generales	Push-pull	0	0		0 o 1
	Open-drain	0	1		0 o 1
Salida de Función Alternativa	Push-pull	1	0		NI
	Open-drain	1	1		NI
	Analog	0	0		NI
Entrada	Flotante	0	1		NI
	Pull-down	1	0		0
	Pull-up	1	0		1

Modo	MODE[1:0]
No corresponde (pasa a Entrada)	00
Máxima salida 10 MHz	01
Máxima salida 2 MHz	10
Máxima salida 50 MHz	11

MODE[1:0]: Entrada (00) o salida (depende la velocidad)
CNF[1:0]: define la configuración fina de cada modo

[CNF][MODE]

Cada puerto tiene 16 pines, para configurar un solo pin se utilizan los 4 bits anteriores (MODE, CNF), pero 16×4 es 64, como el cortex M3 es de 32 bits, partimos el registro a la mitad.

GPIOx_CRL del 0 al 7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
7				6				5				4				3				2				1				0			

GPIOx_CRH del 8 al 15

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
15				14				13				12				11				10				9				8			

Configuremos los 2 periféricos anteriores

ADC $\rightarrow$ PA1 y PA2 $\rightarrow$ Analog, UART1_Tx $\rightarrow$ PA9 $\rightarrow$ AF PP
UART1_Rx $\rightarrow$ PA10 $\rightarrow$ IF 0 IPO

// Analog es : 0000, el PA1 es [7:4] y el PA2 es [11:8] entonces

GPIOA $\rightarrow$ CRL $\&= \sim(0xF < 4) | (0xF < 8);$

// Tx $\rightarrow$ PA9 $\rightarrow$ [7:4] $\rightarrow$ 1 0 1 1, Rx $\rightarrow$ PA10 [11:8] 0 1 0 0

GPIOA $\rightarrow$ CRL $\&= \sim(0xF < 4);$

GPIOA $\rightarrow$ CRL $|= (0xB < 4);$

GPIOA $\rightarrow$ CRL $\&= \sim(0xF < 8);$

GPIOA $\rightarrow$ CRL $|= (0x4 < 8);$

3) Capa de protocolo

4) Capa de operativa

Excepciones

Cualquier evento no programado que altere el flujo normal de ejecución de las instrucciones.

Pueden ser internas, externas, sincrónicas, asincrónicas, son similares a una función en: guardar la dirección de retorno, saltar a la dirección del código que atiende a la excepción, restaurar los valores

- Mapa Mental Tc 11